CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 33: Tool Distribution in Multi-Agent Systems

> **Exam objective:** Configure tool distribution in multi-agent systems by assigning each subagent only the tools required for its designated role, reducing decision complexity and preventing out-of-role tool invocations.

Q1.

You are designing a multi-agent code-review system with separate agents for log analysis, source inspection, and deployment validation. What tool distribution principle best aligns with the objective?

A. Give every agent all tools so they can help each other dynamically B. Assign each agent only the tools needed for its role, such as logs access for the log analyst and repository read tools for the code reviewer C. Remove tools entirely and rely on model reasoning D. Give the deployment validator write access to source files in case it spots bugs

Answer: B **Explanation:** Role-specific tool assignment reduces unnecessary decision branches and constrains agents away from irrelevant or risky actions. A and D expand authority beyond role needs, while C eliminates operational capability entirely.

Q2.

A summarization subagent only needs to combine findings from other agents into a final report. Which tool set is most appropriate?

A. Full repository edit tools, shell, network, and production credentials B. No tools beyond access to the upstream findings it must summarize C. Shell access so it can investigate anything missing D. Database write access in case it wants to restructure the data

Answer: B **Explanation:** A pure summarizer should have the minimum capability necessary: consuming prior outputs and writing a summary. Granting broad investigative or write tools increases complexity and the chance of out-of-role actions.

Q3.

In a triage workflow, a subagent tasked only with locating candidate files keeps trying to run tests and inspect deployment scripts. What is the clearest architectural fix?

A. Increase the model size B. Rewrite the system prompt to say "stay focused" C. Restrict the subagent's tools to file-discovery capabilities needed for locating candidate files D. Add even more

tools so it can finish the whole task itself

Answer: C **Explanation:** The strongest control is capability design: if the agent only needs discovery tools, it should not be able to run tests or deployment commands at all. Prompting alone is weaker, and adding tools worsens the problem.

Q4.

Why does limiting tools per subagent often improve agent performance even when all agents are trustworthy?

A. It reduces decision complexity and lowers the chance of unproductive tool selection B. It guarantees lower token usage in every case C. It makes agents deterministic D. It removes the need for orchestration

Answer: A **Explanation:** Fewer available tools mean fewer branching choices, which often improves focus and efficiency. The other options claim guarantees that are not generally true.

Q5.

You have a documentation-only subagent that must update markdown files but should never execute commands. Which configuration is best?

A. Give it read, write, and shell tools so it can verify everything itself B. Give it only the file read/write tools needed for documentation changes C. Give it only shell access because shell can do anything D. Give it network access to external search providers instead of local file tools

Answer: B **Explanation:** The agent's role is bounded to documentation file edits, so it should only receive file tools required for that task. Broader command or network access expands risk and unnecessary choice without adding role value.

Q6.

An orchestrator assigns one subagent to inspect SQL migrations and another to review frontend components. What is the main risk of giving both agents identical full tool access?

A. They will always produce identical findings B. They may wander into each other's domains, duplicate effort, or invoke unnecessary tools unrelated to their assignments C. They will be unable to share context D. They will stop using any tools at all

Answer: B **Explanation:** Full shared access makes it easier for agents to drift, overlap, and spend cycles on irrelevant actions. The other outcomes are not the primary architectural concern addressed by scoped tool distribution.

Q7.

A security-review subagent needs to inspect code and dependency manifests but should not modify files. Which tool distribution is best?

A. Read-only repository tools without edit capability B. Full edit and shell tools because security work is important C. Database admin tools so it can track findings D. No tools; security review

should be intuition-based

Answer: A **Explanation:** A read-only tool set is sufficient and safer for a review-only role. Adding modification or unrelated administrative tools increases attack surface and out-of-role behavior.

Q8.

Which scenario is the strongest example of an out-of-role tool invocation?

A. A search subagent uses filename matching to locate `*.yaml` files B. A report-writing subagent calls production deployment tooling to verify a sentence in its summary C. A test subagent runs the project's existing test suite D. A parser subagent reads the input schema before extracting fields

Answer: B **Explanation:** A reporting agent should not be operating deployment tooling; that is a clear role violation and an avoidable risk. The other actions are aligned with each subagent's described responsibility.

Q9.

An architect claims, "If prompts are strong enough, fine-grained tool assignment is unnecessary." What is the best rebuttal?

A. Strong prompts and scoped tools address different layers; capability limits enforce role boundaries even when prompts are imperfect B. Tool scoping only matters for cost, not behavior C. Prompts are irrelevant once tools exist D. Subagents should never have prompts at all

Answer: A **Explanation:** Prompting guides intent, while tool assignment constrains actual capability; both matter, but scoped tools provide a stronger enforcement mechanism. The other statements are overly absolute or incorrect.

Q10.

You are building a three-agent pipeline: discover files, inspect code, and synthesize recommendations. Which distribution best fits the objective?

A. All three agents receive the same full tool set for flexibility B. Discovery gets search tools, inspection gets read-focused tools, synthesis gets only the findings it must combine C. Discovery gets shell and deployment access, inspection gets no tools, synthesis gets edit access to source code D. Each agent receives every tool except the one it is most likely to need

Answer: B **Explanation:** B assigns minimum necessary capability at each stage, which keeps agents focused and reduces accidental tool misuse. The other options misalign tools to roles or maximize complexity instead of minimizing it.

Q11.

What is the most likely effect of giving a narrow specialist subagent a very large menu of irrelevant tools?

A. Better specialization through optionality B. Higher decision overhead and more opportunities for inefficient or incorrect tool calls C. Guaranteed faster execution D. Elimination of orchestration

bugs

Answer: B **Explanation:** Too many irrelevant tools create extra branching and distraction, which can degrade agent behavior. More options do not automatically improve specialization, speed, or orchestration quality.

Q12.

A subagent's role is to validate whether generated JSON conforms to a schema already provided in context. Which tool configuration is most appropriate?

A. No external tools if validation can be done from the provided schema and payload B. Shell, network, and repository editing tools C. Database admin tools only D. Deployment credentials plus schema access

Answer: A **Explanation:** If the task can be completed from context alone, no additional tools should be granted. Unnecessary tools increase complexity without improving task completion.

Q13.

In a customer-support workflow, one subagent classifies tickets while another drafts responses. Why might you intentionally deny the classifier access to outbound communication tools?

A. To force it to be more creative B. Because the classifier's role ends at categorization, and response-sending capability would be outside its designated function C. Because classifiers should never read tickets D. Because communication tools make classification impossible

Answer: B **Explanation:** Role boundaries should reflect the workflow stages, and the classifier does not need communication capability to do its job. Denying that tool prevents accidental or premature outward actions.

Q14.

Which metric would most directly indicate that improved tool distribution is helping a multi-agent system?

A. Increased number of total tools available across the system B. Reduced frequency of irrelevant tool invocations by subagents C. Longer prompts for each agent D. More overlap between agents' responsibilities

Answer: B **Explanation:** The objective specifically targets reducing out-of-role and unnecessary tool calls, so fewer irrelevant invocations is the clearest success signal. The other metrics either move in the wrong direction or do not measure the goal directly.

Q15.

When should you deliberately give two different subagents access to the same tool?

A. Always, for consistency B. Never, because shared tools are inherently insecure C. Only when that tool is genuinely required by both roles, despite otherwise distinct responsibilities D. Only if the orchestrator cannot decide which agent should use it

Answer: C **Explanation:** Shared access is appropriate when multiple roles truly need the same capability, but it should still be justified by role requirements. Blanket sharing or blanket denial ignores the principle of least necessary capability.